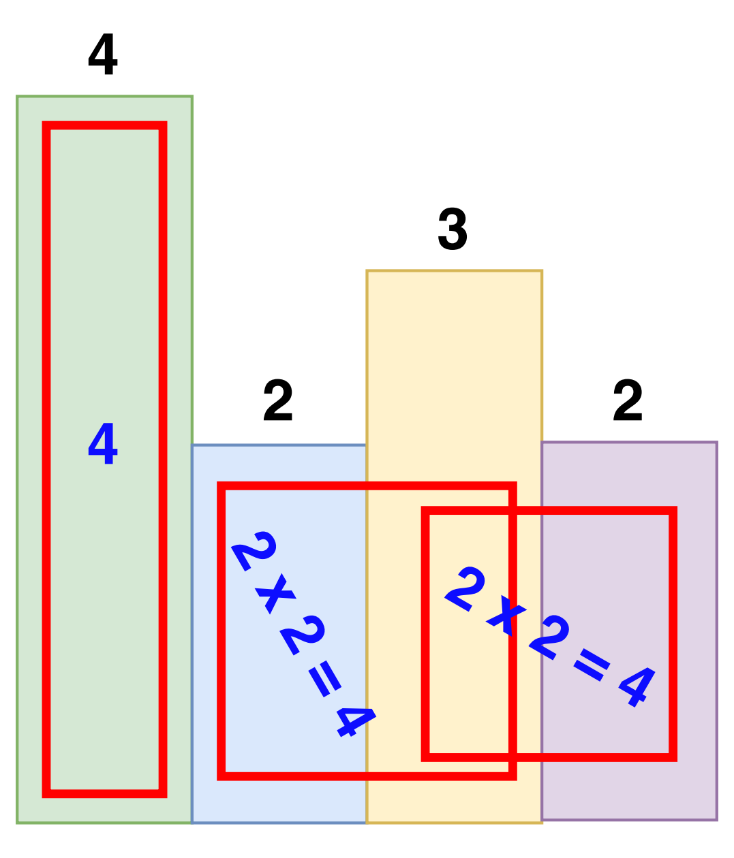
Solution: Largest Rectangle in Histogram

Problem Statement

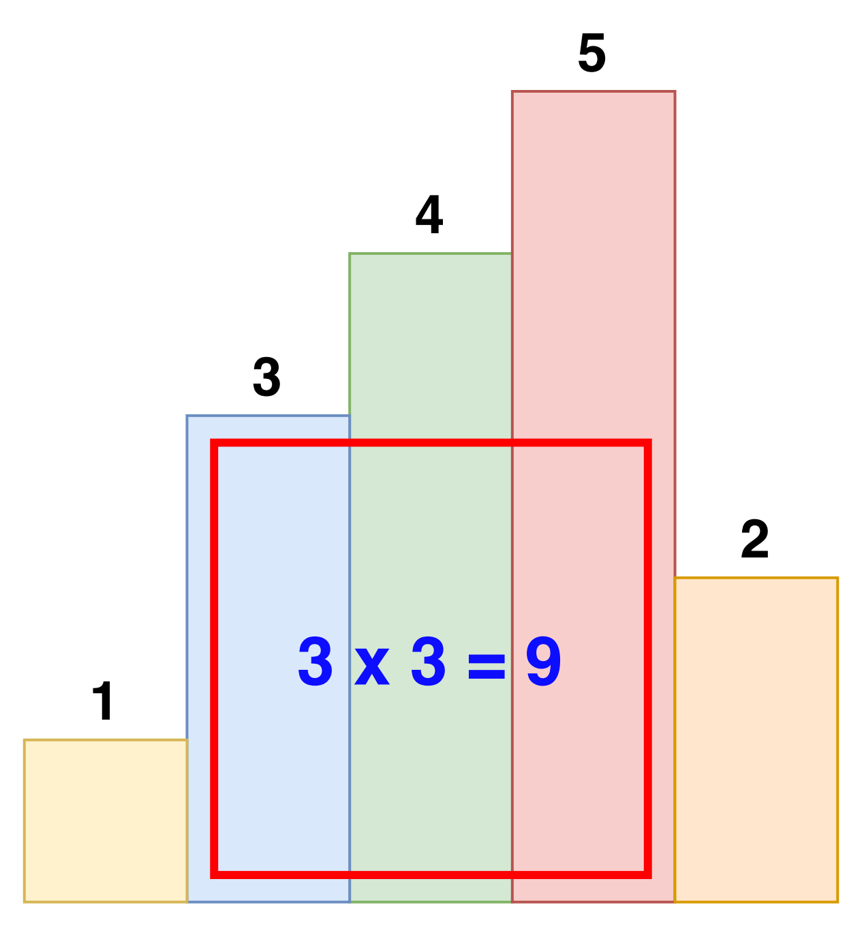
Given an array of integers `heights`, where `heights[i]` represents the height of the histogram's bar, return the area of the `largest rectangle` in the histogram. Each bar has width equal to `1`.

Examples

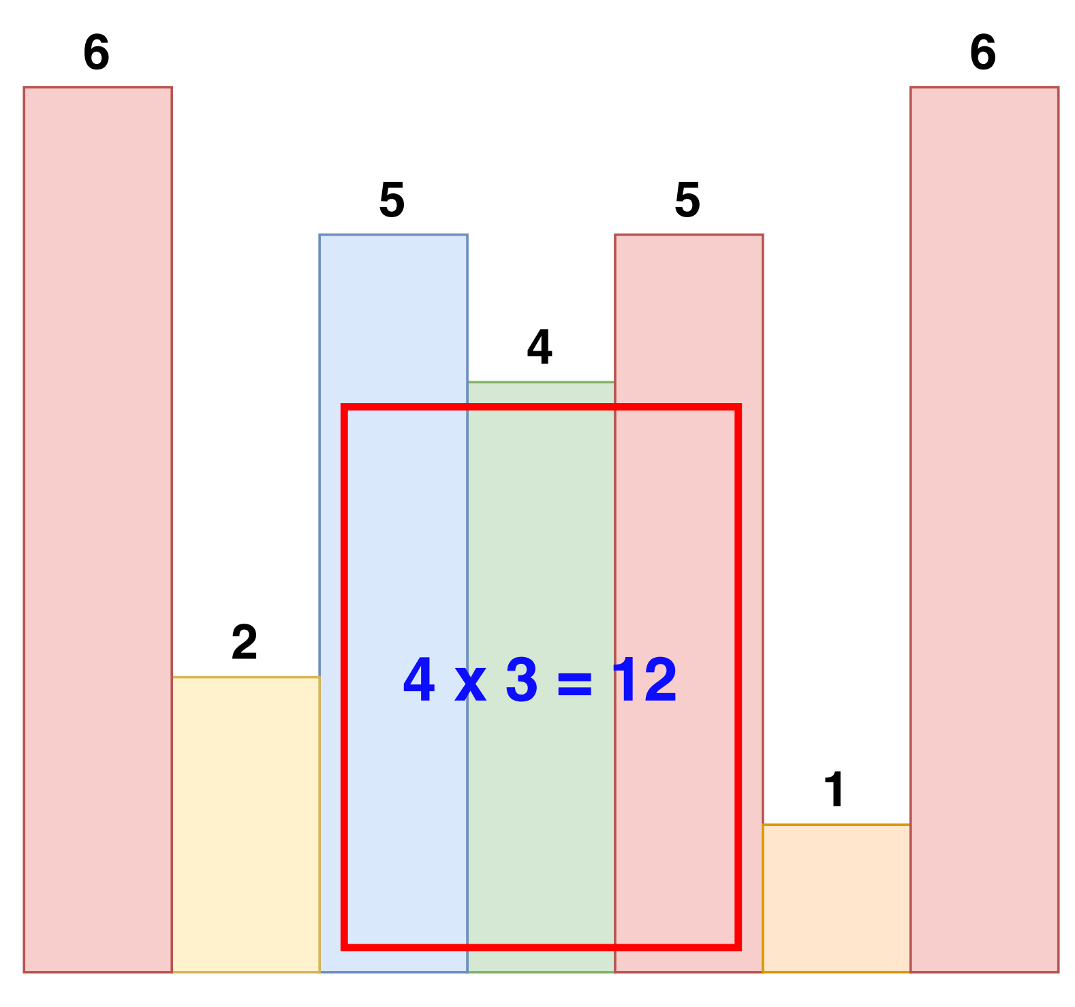
- **Example 1**:
  - **Input**: `[4, 2,0,3,2]`



- **Expected Output**: `4`
- **Justification**: The largest rectangle can be formed by using the bar with height `4` only or `4` and `2` or `3` and `2`, covering a total area of `2x2=4`.
- **Example 2**:
  - **Input**: `[1,3,4,5,2]`



- **Expected Output**: `9`
- **Justification**: The largest rectangle can be drawn from the second bar to the fourth bar, having height `3`, resulting in an area of `3x3=9`.
- **Example 3**:
  - **Input**: `[6,2,5,4,5,1,6]`



- **Expected Output**: `12`
- **Justification**: The maximum rectangular area is formed by the bars of heights `5`, `4`, `5`, which is equal to `4x3=12`. This rectangle spans from the third to the sixth bar.

Solution

To solve this problem, the key approach is to utilize a stack to keep track of bars that are potentially part of the largest rectangle. We traverse each bar in the histogram and use the stack to maintain bars in ascending order of height. When we encounter a bar that is shorter than the bar at the top of the stack, it indicates the end of a potentially largest rectangle involving the top-of-stack bar. By calculating the area of the rectangle with the stack's top bar as the smallest bar and comparing it with the maximum area found so far, we ensure that we find the largest rectangle.

This method is effective because it allows us to dynamically adjust to the changing potential for the largest rectangle as we iterate through the bars. It combines the efficiency of a single pass through the data with the ability to backtrack and calculate areas as soon as we know a rectangle's bounds. This blend of forward-looking and immediate calculation ensures we efficiently find the largest possible rectangle.

Step-by-step Algorithm

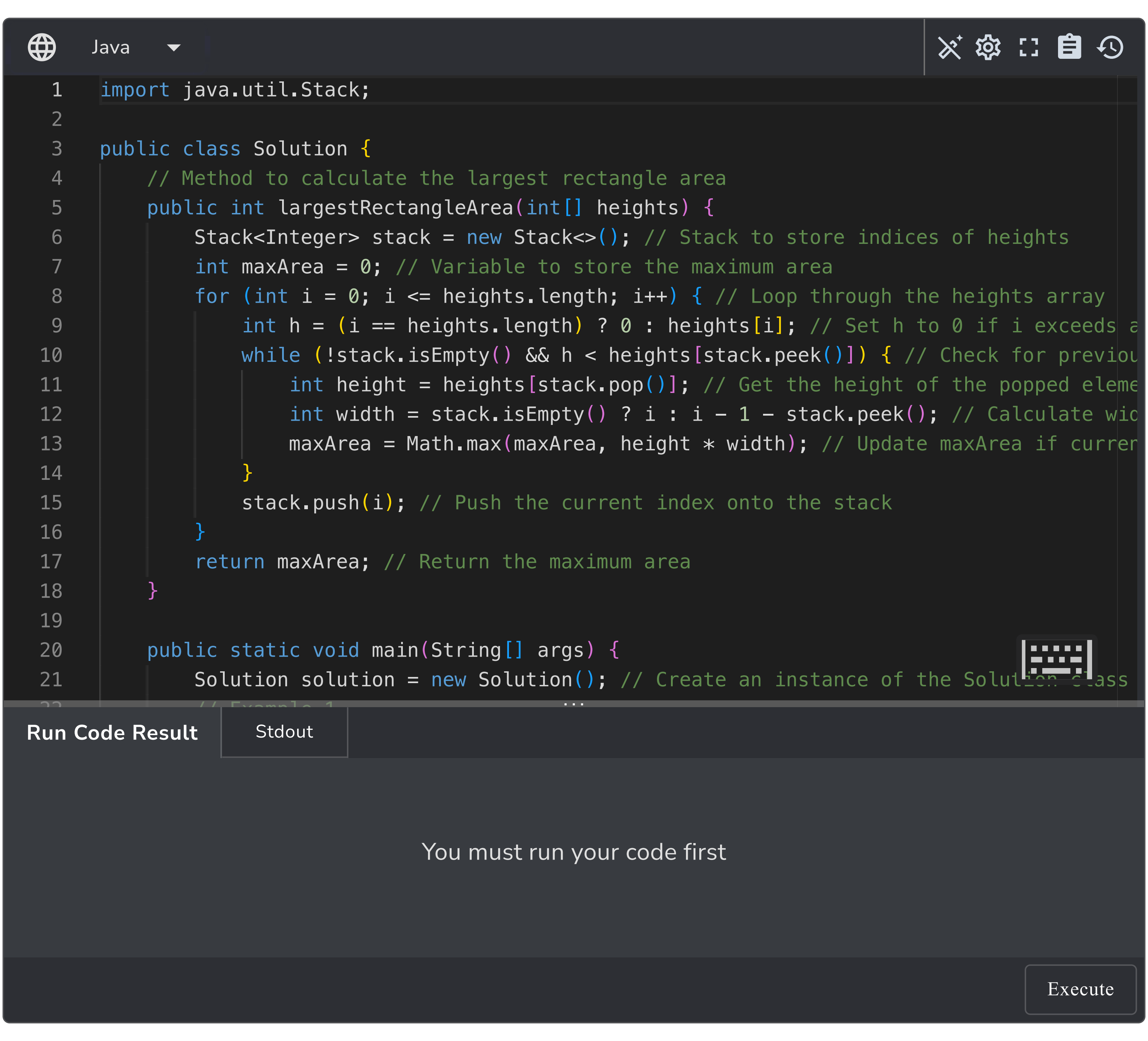
- Initialize an empty stack to keep track of the indices of bars.
- Iterate through each bar in the histogram.
  - While the stack is not empty and the current bar's height is less than the height of the bar at the top of the stack:
    - Pop the top of the stack. Let this be `topIndex`.
    - Calculate the area of the rectangle using the height of the popped bar. If the stack is empty, the width is the current index `i`; otherwise, it's `i - stack.peek() - 1`.
    - Update the maximum area if necessary.
  - Push the current index onto the stack.
- Return the maximum area found.

Algorithm Walkthrough

Let's consider the input `[6,2,5,4,5,1,6]`:

- **Step 1**: Start with an empty stack and `maxArea = 0`.
- **Step 2**: Iterate through each bar's height:
  - **At index 0** (`height=6`):
    - Push 0 onto the stack.
    - `maxArea` remains `0`.
  - **At index 1** (`height=2`):
    - Since `6 > 2`, pop index 0 from the stack. Calculate area: `6 * (1 - 0) = 6`. Update `maxArea` to `6`.
    - Push index 1 onto the stack.
  - **At index 2** (`height=5`):
    - Push index 2 onto the stack since `2 < 5`. `maxArea` remains `6`.
  - **At index 3** (`height=4`):
    - Since `5 > 4`, pop index 2 from the stack. Calculate area: `5 * (3 - 1 - 1) = 5`. `maxArea` remains same.
    - Push index 3 onto the stack.
  - **At index 4** (`height=5`):
    - Push index 4 onto the stack since `4 < 5`. `maxArea` remains `6`.
  - **At index 5** (`height=1`):
    - Since `5 > 1`, pop index 4 from the stack. Calculate area: `5 * (5 - 3 - 1) = 5`. `maxArea` remains `6`.
    - Since `4 > 1`, pop index 3 from the stack. Calculate area: `4 * (5 - 1 - 1) = 12`. Update `maxArea` to `12`.
    - Since `2 > 1`, pop index 1 from the stack. Calculate area: `2 * (i) = 2 * (5) = 10`. `maxArea` remains same `12`.
    - Push index 5 onto the stack.
  - **At index 6** (`height=6`):
    - Since `1 < 6`, push index 6 onto the stack. `maxArea` remains `12`.
- **Final** `maxArea` **is** `12`.

Code



```java
import java.util.Stack;

public class Solution {
    // Method to calculate the largest rectangle area
    public int largestRectangleArea(int[] heights) {
        Stack<Integer> stack = new Stack<>(); // Stack to store indices of heights
        int maxArea = 0; // Variable to store the maximum area
        for (int i = 0; i <= heights.length; i++) { // Loop through the heights array
            int h = (i == heights.length) ? 0 : heights[i]; // Set h to 0 if i exceeds a
            while (!stack.isEmpty() && h < heights[stack.peek()]) { // Check for previou
                int height = heights[stack.pop()]; // Get the height of the popped eleme
                int width = stack.isEmpty() ? i : i - 1 - stack.peek(); // Calculate wid
                maxArea = Math.max(maxArea, height * width); // Update maxArea if curren
            }
            stack.push(i); // Push the current index onto the stack
        }
        return maxArea; // Return the maximum area
    }

    public static void main(String[] args) {
        Solution solution = new Solution(); // Create an instance of the Solution class
```

You must run your code first

Complexity Analysis

Time Complexity

- $O(n)$: The algorithm traverses the histogram (array of bar heights) exactly twice in the worst case. The first traversal is the main loop where each bar is examined once. The second traversal occurs implicitly through stack operations. This leads to the linear time complexity.

Space Complexity

- $O(n)$: The space complexity is determined by the stack that is used to store indices of bars. In the worst-case scenario, all bars are pushed onto the stack, requiring a space proportional to the number of bars.